Learning Agents: Technical Foundations, Architectures, Algorithms, and Frontiers

The term **learning agent** refers to an AI system that not only perceives, plans, and acts but also improves its own behavior from experience, feedback, and interaction with tools or environments. In the current large language model (LLM) era, a learning agent is best understood as an extension of the AI agent concept: it combines an LLM "brain" with memory, planning, tool use, and a closed-loop learning mechanism that updates its policy, knowledge, or memory over time (Volcano Engine, 2024). This report provides a technical synthesis of learning agents across definitions, core architectures, learning mechanisms, application scenarios, frontier progress, challenges, and future directions.

1. Definition and Core Principles

An autonomous agent has classically been defined as a system situated within an environment that senses that environment and acts on it over time in pursuit of its own agenda (Volcano Engine, 2024). Modern LLM-based agents extend this idea by using a language model as the central reasoning engine. The most widely adopted formulation is:

> ***Agent = LLM + memory + planning + tools + action***

This formulation was popularized by OpenAI researcher Lilian Weng and is now a standard reference architecture in the Chinese and international agent literature (九卷，2026；Prompt Engineering Guide, n.d.）.

A **learning agent** goes one step further: it does not merely execute a fixed reasoning loop but also modifies its future behavior through one or more learning signals. These signals may include human feedback, environmental rewards, execution traces, retrieved memory, or fine-tuning on successful trajectories. This distinction is important. As the technical survey by stardsd notes, the field has moved from prompt-based scaffolding toward **agentic reinforcement learning** and end-to-end agent training, making learning a first-class property of modern agents (stardsd, 2026）.

The core principles of a learning agent can be summarized as follows.

Principle	Description
Autonomy	The agent selects actions and plans without step-by-step human control.
Perception–action loop	The agent observes tool results, environment states, or user feedback, then updates its next action.
Memory persistence	The agent retains short-term context and long-term knowledge across sessions.
Adaptivity	The agent modifies its strategy, memory, or weights based on experience.
Tool-grounded action	The agent interacts with external systems through APIs, code interpreters, search, or protocol-based tools.

A learning agent therefore differs from a simple chatbot in that it has a persistent state and a mechanism for self-improvement. This is especially relevant for production deployments, where agents must handle long-horizon tasks, recover from errors, and adapt to changing user preferences.

2. Core Architecture of Learning Agents

2.1 Reference Agentic Architecture

A modern agentic AI architecture connects a language model with tools, memory, and workflow orchestration. According to Kimi Resources, the architecture generally includes:

- **Reasoning, planning, and task decomposition**: The layer that converts high-level goals into actionable steps.
- **Tool and action layer**: The interface that allows the agent to check or affect external systems.
- **Memory, context, and knowledge**: Working memory for the current plan, long-term memory for preferences, and retrieval for facts.
- **Orchestration and coordination**: The control loop that manages how work flows across components (Kimi Resources, 2026).

This layered view is consistent with the widely used engineering architecture in which an agent receives user input, perceives context, plans, calls tools, observes results, and finally produces an answer or action (九卷， 2026).

2.2 Nine Core Technologies in Agent Architecture Design

A practitioner-oriented CSDN guide identifies nine core technical modules for designing AI agent architectures: AI Agent, Agentic AI, WorkFlow, RAG, Fine-tuning, Function Calling, MCP, A2A, and AG-UI (CSDN, 2026). These modules map directly to the learning agent stack.

Technology	Function in a learning agent
AI Agent	The autonomous entity that perceives, decides, and acts.
Agentic AI	The architectural style enabling repeated reasoning and action toward a goal.
WorkFlow	Deterministic orchestration of subtasks and control flow.
RAG	Retrieval-augmented generation for external knowledge injection.
Fine-tuning	Model adaptation to new domains, formats, or agent trajectories.
Function Calling	Structured invocation of external APIs and tools.
MCP	A standardized protocol for connecting agents to tools and data sources.
A2A	An agent-to-agent communication protocol for multi-agent workflows.
AG-UI	The user-facing interface layer for agent interaction and feedback.

Source: Adapted from CSDN, 2026.

These technologies are not independent. In a learning agent, RAG feeds memory, function calling enables tool use, fine-tuning updates model weights, and MCP/A2A standardize integration across tools and agents. A production learning agent typically combines several of these modules simultaneously.

2.3 Memory Architecture

Memory is central to learning. A learning agent must decide what to remember, how to retrieve it, and when to forget. The survey by stardsd identifies three major memory approaches:

1. **External/parametric memory**: vector databases or graph databases with retrieval.
2. **In-context memory**: compression, summarization, and context paging.

3. **Parameterized memory**: fine-tuning to write experience into model weights (stardsd, 2026).

Representative systems include MemGPT, which borrows virtual memory paging from operating systems to extend context; Mem0, which uses a two-stage extraction–update memory pipeline; and A-MEM, which enables agents to build linked, evolving memory indexes (stardsd, 2026). From a learning perspective, memory is not only a storage mechanism but also a source of experience for future decisions.

2.4 Multi-Agent Architectures

Learning agents can also be organized into multi-agent systems. Early frameworks such as AutoGen, MetaGPT, and ChatDev demonstrated role-based collaboration in software engineering and other tasks (stardsd, 2026). However, later research cautioned that some multi-agent gains came from weak single-agent baselines rather than inherent collaboration benefits. The Berkeley MAST taxonomy identified 14 failure modes across three categories: specification defects, inter-agent coordination failures, and verification gaps (stardsd, 2026).

For learning agents, the practical implication is that multi-agent structures should be adopted only when tasks are clearly parallelizable or require independent verification. Anthropic's engineering experience showed that orchestrator–worker architectures help in information-collection tasks but may cost approximately 15 times more tokens than single-agent approaches (stardsd, 2026). Thus, architecture choice is not neutral; it affects both learning efficiency and operating cost.

3. Learning Mechanisms and Algorithms

3.1 In-Context Learning and Prompt-Based Scaffolding

Before end-to-end training became dominant, learning in agents was mostly achieved through structured prompting and in-context examples. ReAct introduced the reasoning–action interleaving paradigm, while Reflexion used verbal self-criticism as a reinforcement signal. Tree of Thoughts and LATS applied Monte Carlo tree search to LLM decision-making, enabling agents to explore and backtrack (stardsd, 2026).

These methods are still useful. They allow rapid prototyping and do not require model weight updates. However, the survey notes a clear trend: as base models become stronger, complex prompt scaffolding may yield diminishing returns, and over-orchestration can even reduce performance

（stardsd, 2026). For learning agents, this suggests that prompt-based learning is a lightweight complement to, but not a replacement for, model-level learning.

3.2 Supervised Fine-Tuning

Fine-tuning is one of the nine core technologies identified by the CSDN guide (CSDN, 2026). In the context of learning agents, fine-tuning is used to:

- Adapt a general model to a specialized domain.
- Imitate successful agent trajectories.
- Improve instruction following and structured tool calling.
- Compress long-term knowledge into model weights.

Fine-tuning is especially important when an agent must consistently produce a particular output format or follow strict operational procedures. However, it can be static unless combined with additional feedback loops.

3.3 Reinforcement Learning and Agentic RL

The most significant learning-algorithm shift in recent years is **agentic reinforcement learning (agentic RL)**, also called RL with verifiable rewards (RLVR). Instead of only prompting a frozen model, agentic RL trains the model in interactive environments using task outcomes as rewards（stardsd, 2026）.

Several representative works illustrate this direction:

- **SWE-agent** showed that the design of the agent–computer interface can substantially release model capability on software engineering tasks.
- **SWE-smith** synthesizes training tasks by deliberately injecting bugs into codebases, addressing the scarcity of real agent data.
- **WebRL** uses a self-evolving curriculum and outcome reward model to train web agents.
- **AgentGym** and **RAGEN** provide general multi-environment RL infrastructure.
- **DeepResearcher** and **Search-R1** train agents to retrieve and reason simultaneously in real internet environments (stardsd, 2026）.

Reasoning models such as OpenAI o1 and DeepSeek-R1 represent the model-side foundation of this trend. They show that long chain-of-thought reasoning and reinforcement learning can improve multi-step performance dramatically (stardsd, 2026). A learning agent built on such reasoning models can better plan, self-correct, and allocate test-time compute.

3.4 Memory-Based Learning

Memory is not only a storage component but also a learning channel. MemGPT introduced hierarchical memory management to extend context across long sessions. Mem0 proposed an extraction–update memory pipeline that outperforms full-context baselines in benchmark settings. A-MEM allows agents to create linked notes and evolving indexes, similar to a Zettelkasten note-taking system (stardsd, 2026).

From a learning perspective, these systems enable the agent to accumulate experience without changing model weights. They are especially useful for personalization, multi-session tasks, and long-horizon projects.

3.5 Summary of Learning Mechanisms

Mechanism	Strengths	Limitations
Prompt scaffolding & in-context learning	Fast, no training required	Limited by context window and model priors
Supervised fine-tuning	Stable domain adaptation	Static unless retrained
Agentic RL / RLVR	End-to-end policy improvement	Sparse rewards, high compute, environment design
Memory-based learning	Flexible, low-cost experience retention	Retrieval quality and memory governance issues

Adapted from stardsd, 2026; CSDN, 2026.

4. Typical Application Scenarios

Learning agents have moved beyond demos into measurable deployment scenarios. The following table summarizes representative domains.

Domain	Example	Key Learning Element
Education	A chemistry support agent that checks understanding and estimates student knowledge levels (53AI, 2024)	Student feedback, question logs, knowledge estimation

Domain	Example	Key Learning Element
Customer service	An agent that answers questions, queries inventory, and processes orders (Volcano Engine, 2024)	Interaction history, tool results
Programming assistance	Code generation, execution, debugging, and performance optimization (Volcano Engine, 2024)	Execution feedback, code traces
Data analysis and reporting	Inventory quantile calculation and PDF report generation (Baidu International, 2025)	Numeric verification, workflow feedback
ESG report generation	Amazon Bedrock–based agent using LangGraph Map-Reduce and quality-evaluation loops (AWS, 2025)	Expert evaluation, iterative quality improvement
Research report generation	NVIDIA Nemotron–based agent performing research, outlining, section writing, and compilation (NVIDIA, 2025)	Web search results, outline changes, section-level revision

The AWS ESG example is particularly instructive. The agent uses a closed loop of "generation – evaluation – iterative optimization" for each topic report. It combines vector database retrieval, LangGraph workflows, and LLM-based quality evaluation to ensure that outputs meet a predefined threshold before proceeding (AWS, 2025). This is a practical example of a learning loop embedded in a production system.

Similarly, the Baidu International case shows how agent mode transforms LLMs from direct answer generators into code generators and task executors. The agent parses requirements, plans tasks, calls Python tools, validates results, and produces reports. In one manufacturing deployment, daily report generation time dropped from four hours to eight minutes, with human review reduced by 70% and error rate below 0.3% (Baidu International, 2025). These numbers illustrate the concrete operational value of learning agents when feedback and validation loops are properly designed.

5. Frontier Progress

5.1 From Reasoning Models to Agentic Training

A major frontier shift has occurred from static LLM prompting to **reasoning-time compute scaling** and **agentic RL**. OpenAI o1 and DeepSeek-R1 demonstrated that long chain-of-thought reasoning with reinforcement learning can significantly improve multi-step reasoning. This has been described as a move from pre-training scaling to test-time compute scaling (stardsd, 2026). For learning agents, this means the underlying model is now more capable of planning, self-correction, and uncertainty-aware decision-making.

5.2 Computer Use, GUI Agents, and Deep Research

New agent categories have emerged. **Computer use agents** interact with graphical user interfaces, **GUI agents** perform cross-application tasks, and **deep research agents** automate multi-step retrieval, reading, synthesis, and report writing (stardsd, 2026). OpenAI Deep Research improved performance on Humanity's Last Exam from 26.6% to 67.3%, highlighting the value of long-horizon autonomous research (stardsd, 2026). These are learning-agent applications because the agent must adjust its search strategy, verify evidence, and revise outputs during execution.

5.3 Standardized Protocols: MCP and A2A

Tool integration has become standardized. Anthropic released the **Model Context Protocol (MCP)** in November 2024, and it was adopted by OpenAI and Google DeepMind within six months (stardsd, 2026). MCP provides a unified way to connect agents with data sources and tools. A2A enables agent-to-agent communication. AG-UI completes the stack by providing a standard user-facing interface layer (CSDN, 2026). For learning agents, protocols reduce integration cost and enable more reproducible tool-use training.

5.4 Evaluation and Measurement

Evaluation has become a central research problem. Key benchmarks include SWE-bench Verified for coding agents, OSWorld for computer use, GAIA for multi-step assistant tasks, and τ-bench for tool-use dialogues (stardsd, 2026). METR's **task time horizon** metric measures how long an agent can autonomously complete a task, giving a quantitative view of agent capability growth.

However, benchmark saturation and contamination are serious concerns. For example, SWE-bench Verified human expert baseline is approximately 42%, and the best systems exceeded 75% by 2025,

pushing the community toward harder benchmarks such as SWE-bench Pro (stardsd, 2026). A learning agent designed for production should therefore be evaluated on dynamic or private tasks, not only public benchmarks.

6. Technical Challenges

Despite rapid progress, learning agents face several unresolved challenges.

6.1 Reliability and Long-Horizon Performance

Long-horizon autonomous execution remains difficult. The failure taxonomy from the multi-agent literature identifies numerous failure modes, including specification defects, coordination failures, and verification gaps (stardsd, 2026). In GUI agents, the main failures are perception localization errors and error accumulation over long sequences (stardsd, 2026). For learning agents, these failures provide useful training signals, but they also make deployment riskier without human oversight.

6.2 Sparse Rewards and Credit Assignment

Agentic RL suffers from sparse rewards, credit assignment across multiple interaction steps, policy collapse, and environment distribution shift (stardsd, 2026). Unlike single-turn tasks, a long agent trajectory may contain many steps, only some of which contribute to final success. This makes naive reward propagation inefficient. The field is actively exploring outcome reward models, process supervision, and curriculum learning to address this.

6.3 Cost and Token Economics

Learning agents, especially multi-agent systems, can be expensive. Anthropic reported that an orchestrator–worker multi-agent architecture improved quality and latency but consumed roughly 15 times more tokens than a single agent (stardsd, 2026). For production systems, this means learning agents must include cost-aware routing, caching, and possibly smaller specialized models for subtasks.

6.4 Safety, Security, and Governance

Learning agents introduce new safety risks. Prompt injection, tool permission abuse, memory privacy violations, and unverified external actions all require formal defenses and audit mechanisms. The production engineering guide identifies safety, compliance, governance, and

human-in-the-loop controls as essential layers (CSDN Agent, 2026). Memory governance is also becoming a research issue: who can write to agent memory, how long memories persist, and how to comply with privacy regulations are open questions (stardsd, 2026).

6.5 Over-Orchestration and Scaffolding Debt

As base models improve, complex prompting and multi-module scaffolding may no longer be necessary or beneficial. The survey notes that some gains from 2023–2024 scaffolding disappeared as model reasoning improved, and over-orchestration could reduce performance (stardsd, 2026). For learning agents, this suggests a design principle: **scaffolding should target model weaknesses such as memory, verification, and rollback, rather than replace model reasoning**.

7. Future Trends and Outlook

7.1 End-to-End Agentic RL Will Become the Default Training Method

The most important trend is the shift from prompt engineering to end-to-end training in interactive environments. With infrastructure such as SWE-smith, AgentGym, WebRL, and RAGEN, training data and environments are becoming more accessible (stardsd, 2026). Future learning agents will likely be continuously fine-tuned on successful trajectories, human corrections, and environment rewards, rather than remaining static after deployment.

7.2 Memory and Self-Evolution Will Become More Governed

Memory systems will continue to improve, but the focus will shift from raw capacity to governance, retrieval quality, and privacy compliance. Self-evolving agents that update parameters, tools, and memory dynamically are an active research direction (stardsd, 2026). However, uncontrolled self-modification raises safety and reliability concerns, so future systems will need explicit update permissions, rollback mechanisms, and audit trails.

7.3 Standardized Protocols Will Reduce Integration Friction

MCP, A2A, and AG-UI are becoming de facto standards (CSDN, 2026; stardsd, 2026). This standardization will make learning agents more portable across tools, platforms, and vendors. It will also enable more consistent tool-use training and evaluation.

7.4 Multi-Agent Use Will Become More Selective

The community is converging on a nuanced view: multi-agent systems are useful for breadth-oriented, parallelizable tasks; single agents are better for deep chained tasks (stardsd, 2026). Future learning-agent platforms will likely use hybrid designs, where a central orchestrator activates parallel sub-agents only when task decomposability and verification requirements justify the additional cost.

7.5 Evaluation and Safety Will Become First-Class Infrastructure

Because benchmarks saturate quickly, evaluation will move toward dynamic, private, and long-horizon tasks. Safety engineering will also mature, with formal defenses against prompt injection, tool permission systems, and agent-specific auditing frameworks (stardsd, 2026； CSDN Agent, 2026）.

7.6 A Concrete Assessment

Based on the evidence reviewed here, the most defensible near-term path for learning agents is **not** to add more orchestration layers, but to combine three elements:

1. **A strong reasoning model** with test-time compute capacity.
2. **A lightweight, targeted scaffold** for memory, tool calls, verification, and rollback.
3. **An agentic RL or fine-tuning loop** that converts production interaction data into policy improvements.

This assessment is grounded in the observation that scaffolding-heavy systems lose advantage as models improve, while end-to-end agent training and standardized protocols show the clearest measured gains (stardsd, 2026； CSDN, 2026； CSDN Agent, 2026). A learning agent should therefore be designed as a continuous improvement system, not a one-time prompt-engineering artifact.

8. Conclusion

Learning agents represent the convergence of several previously separate AI threads: autonomous agents, LLM reasoning, memory systems, tool integration, and reinforcement learning. Architecturally, they extend the standard agent stack with more explicit learning loops. Algorithmically, they are moving from static prompt scaffolding to supervised fine-tuning and agentic reinforcement learning. In application, they are already delivering measurable value in

report generation, education, customer service, programming, and research. The main challenges are reliability, sparse-reward RL, cost, safety, and evaluation methodology. The future is likely to favor model-native learning, standardized protocols, governed memory, and selective multi-agent orchestration. Organizations that treat agent development as a continuous learning and evaluation process will be better positioned than those that treat it as a one-time integration task.

References

AWS. (2025). 利用*AWS Bedrock*构建智能报告生成*Agent* [Building an intelligent report generation agent with AWS Bedrock]. Amazon Web Services. https://aws.amazon.com/cn/blogs/china/building-intelligent-report-generation-agent-with-aws-bedrock

Baidu International. (2025). 新一代智能大模型*Agent*模式深度解析：从数据处理到自动化报告生成 [A deep analysis of next-generation LLM agent mode: From data processing to automated report generation]. Baidu Cloud. https://intl.cloud.baidu.com/zh/article/8753301

CSDN. (2026). 一文搞定 *AI* 智能体架构设计的9大核心技术 [Nine core technologies for AI agent architecture design]. CSDN Blog. https://blog.csdn.net/musicml/article/details/148704411

CSDN Agent. (2026). *AI Agent* 开发技术完全学习指南（2026-07 基线版）*part1* [Complete learning guide for AI agent development, July 2026 baseline edition, part 1]. CSDN. https://agent.csdn.net/6a52fd2f662f9a54cb8e8279.html

Kimi Resources. (2026). 智能体式 *AI* 架构：模式与示例 [Agentic AI architecture: Patterns and examples]. Moonshot AI. https://www.kimi.com/resources/agentic-ai-architectures

Liu Cong. (2024). 万字长文讲透智能体丨智能体的架构、探索与应用 [A comprehensive article on agents: Architecture, exploration, and applications]. Volcano Engine Developer Community. https://developer.volcengine.com/articles/7382404827235483699

NVIDIA. (2025). *Build a report generator AI agent with NVIDIA Nemotron on OpenRouter*. NVIDIA Technical Blog. https://developer.nvidia.com/blog/build-a-report-generator-ai-agent-with-nvidia-nemotron-on-openrouter

Prompt Engineering Guide. (n.d.). 大语言模型智能体简介 [Introduction to LLM agents]. https://www.promptingguide.ai/zh/research/llm-agents

stardsd. (2026). *AI Agent* 与大语言模型（*LLM*）最新科研进展技术综述 [A technical survey on recent advances in AI agents and large language models]. 博客园. https://www.cnblogs.com/sddai/p/22181149

Volcano Engine. (2024). 写给小白的大模型应用科普：*AI Agent*篇 [A beginner-friendly introduction to LLM applications: AI Agent]. Volcano Engine Developer Community. https://developer.volcengine.com/articles/7444834061702742042

53AI. (2024). 从国内外10个智能体案例，看*AI Agent*在教育领域的应用 [AI Agent applications in education: 10 cases from home and abroad]. 53AI. https://www.53ai.com/news/AIpeixun/2024101758361.html

九卷. (2026). *LLM*大模型中*AI-Agent*智能体应用开发相关知识介绍 [Introduction to AI agent application development in LLMs]. 博客园. https://www.cnblogs.com/jiujuan/p/19848128